Desenvolvimento de uma Aplicação Web para Consulta de Dados de Servidores do Executivo Federal

Pedro Machado Marcos Freitas Alexsander

Sistemas para Internet II
9 de outubro de 2026

Resumo

Este relatório apresenta o desenvolvimento de uma aplicação full stack para consulta de dados públicos de servidores ativos e aposentados do Poder Executivo Federal. O sistema foi organizado em PostgreSQL, API Flask executada com Gunicorn e interface React, todos coordenados pelo Docker Compose. O texto registra a divisão das atividades, o histórico de commits, o burndown de duas sprints, as funcionalidades entregues, os testes executados e as limitações observadas. As duas sprints encerraram com 115 Story Points obrigatórios concluídos.

Abstract

This report presents the development of a full stack application for querying public data about active and retired employees of the Brazilian Federal Executive Branch. The system combines PostgreSQL, a Flask API served by Gunicorn, and a React interface coordinated by Docker Compose. The report records task allocation, commit history, the burndown of two sprints, implemented features, executed tests, and known limitations. Both sprints were closed with all 115 mandatory Story Points completed.

1 Contexto e objetivo

O projeto teve como objetivo construir uma aplicação full stack para consulta de informações públicas de servidores ativos e aposentados do Poder Executivo Federal. A entrega deveria manter banco, backend e frontend em módulos independentes, iniciar a partir da raiz com Docker Compose e registrar, no relatório final, a divisão diária do trabalho, os commits de cada integrante, o burndown e a análise crítica da equipe.

A equipe foi formada por Pedro, responsável por liderança técnica, infraestrutura, garantia de qualidade e documentação de gestão; Marcos, responsável por banco de dados e backend; e Alex, responsável pelo frontend. A aplicação desktop foi mantida fora do escopo por corresponder ao item opcional da avaliação.

2 Metodologia e organização das sprints

O desenvolvimento foi dividido em duas sprints de 15 dias. A Sprint 1 ocorreu de 12/09 a 26/09/2026 e a Sprint 2 de 27/09 a 11/10/2026, com 12/10 reservado para fechamento antes da

entrega. O Jira registrou atividades, responsáveis, Story Points e andamento. As tasks receberam estimativas de 1 ou 2 pontos e foram ordenadas conforme as dependências entre infraestrutura, dados, implementação, integração, testes e documentação.

As mudanças foram desenvolvidas em branches identificadas pela área e pela chave do Jira. Cada commit correspondeu a uma task e preservou a autoria do responsável. Até duas tasks relacionadas, do mesmo integrante e da mesma área, puderam compartilhar uma branch e uma Pull Request, sem mistura entre seus commits. As integrações foram realizadas em `dev`.

A Sprint 1 totalizou 56 pontos. A Sprint 2 iniciou com 57 pontos e recebeu mais 2 pontos em 09/10 com a inclusão de `LID-07/PROJ-72`, passando a 59. O acompanhamento considerou o quadro do Jira, o histórico Git e as evidências produzidas. Dias sem entrega integrada foram mantidos no registro sem atribuir trabalho ou resultados não comprovados.

Tabela 1: Distribuição final de Story Points por integrante.

Integrante	Sprint 1	Sprint 2	Total
Pedro	16	23	39
Marcos	22	17	39
Alex	18	19	37
Total	56	59	115

3 Arquitetura e execução

A aplicação possui três serviços orquestrados pelo Docker Compose:

- `db`: PostgreSQL 18 Alpine, responsável pelo schema, índices e extensão `pg_trgm`;
- `backend`: Python 3.14, Flask 3.1.3, Gunicorn 26.2.0 e psycopg 3.3.5;
- `frontend`: React 19 com TypeScript, Vite 8 e Tailwind CSS 4.

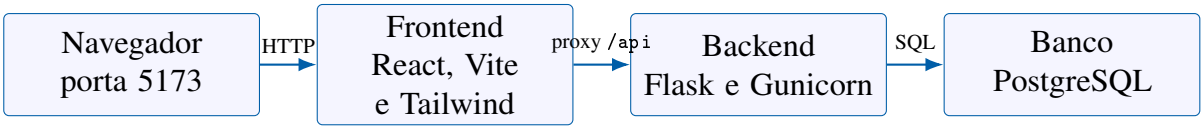


Figura 1: Comunicação entre os módulos da aplicação.

O navegador acessa o frontend pela porta 5173. As requisições iniciadas em `/api` são encaminhadas pelo proxy do Vite ao backend na porta interna 8000. O backend acessa o PostgreSQL pelo nome de serviço `db`; o banco não publica porta para o host. O Gunicorn usa um processo e quatro threads. O Compose limita a API a duas buscas simultâneas e configura timeout de 2.000 ms para as consultas. Banco e backend possuem healthchecks e os três serviços usam a política de reinício `unless-stopped`.

A raiz contém `backend/`, `frontend/`, `db/`, `docs/`, `tests/` e `docker-compose.yml`. O comando de inicialização é:

```
docker compose up --build
```

O arquivo `.env` é opcional na execução local. O Compose fornece valores padrão e `.env.example` registra as variáveis substituíveis de conexão com o banco, limite de concorrência e timeout.

4 Tratamento e organização dos dados

Foram analisados os conjuntos Gestão de Pessoas do Executivo Federal - Aposentados e Gestão de Pessoas do Executivo Federal - Carreiras/Cargos, do Portal Brasileiro de Dados Abertos. A inspeção abrangeu as competências de setembro de 2025 a agosto de 2026. Os recursos de agosto de 2026 estavam indisponíveis e o arquivo de Aposentados de dezembro de 2025 estava vazio.

Os campos utilizados foram nome, cargo, órgão e UF. CPF, matrícula, remuneração e os demais dados pessoais ou funcionais não foram incorporados. A UF não existe no conjunto de Aposentados e permanece nula nesses registros. Os arquivos usam Latin-1 e separação por ponto e vírgula.

O tratamento remove bytes NUL e espaços excedentes; ignora linhas vazias, cabeçalhos e linhas de identificação; trata o delimitador final adicional de Carreiras/Cargos; valida quantidade de colunas e campos obrigatórios; normaliza a UF; rejeita UFs malformadas; remove registros idênticos dentro de cada arquivo; e gera CSV UTF-8 com origem, arquivo e linha do registro.

O modelo relacional contém `servidor`, `cargo`, `orgao` e `uf`. A importação usa uma tabela temporária, insere valores distintos nas tabelas de domínio e cria os relacionamentos em `servidor`. O processo exige tabelas de destino vazias e interrompe a importação se a quantidade inserida divergir da quantidade na tabela temporária. Não existe evidência persistida da contagem final de uma importação pública, portanto nenhum total importado é atribuído neste relatório.

A extensão `pg_trgm` sustenta a similaridade. O nome do servidor usa índice GiST, compatível com a ordenação por distância textual. Cargo e órgão possuem índices trigram GIN, e as chaves estrangeiras de cargo, órgão e UF possuem índices B-tree.

5 Funcionalidades implementadas

5.1 Backend

A API disponibiliza `GET /health` e `GET /api/servidores`. A consulta aceita filtros exatos por nome, cargo, UF e órgão, busca aproximada por nome e combinações entre filtros. A similaridade aceita `limit` entre 1 e 100, com valor padrão 100. A paginação usa `page` e `page_size`, limitada a 100 registros por página, e informa página atual, tamanho, total de registros e total de páginas.

Todas as respostas seguem envelope JSON com `sucesso`, `mensagem` e `dados`; respostas paginadas incluem `paginacao`. A API diferencia registro único, múltiplos registros, ausência de resultados e página inexistente. A UF precisa pertencer à lista de siglas brasileiras, números são validados nos limites definidos e parâmetros desconhecidos são rejeitados.

As consultas usam parâmetros separados do SQL e entradas compatíveis com injeção são recusadas. A API produz falhas controladas para entrada inválida, recurso inexistente, método não permitido, excesso de concorrência, falha interna e timeout. O limite de duas buscas simultâneas retorna HTTP 429 ao excedente, enquanto o cancelamento pelo PostgreSQL é convertido em HTTP 504.

5.2 Frontend

O frontend apresenta nome, modo de busca exata ou por similaridade, limite da similaridade, cargo, UF e órgão. Os filtros podem ser usados isoladamente ou em conjunto. A interface não consulta ao abrir a página e impede envio sem filtro.

Cada consulta usa páginas de dez registros. A navegação oferece ações anterior e próxima e até cinco números de página. A interface diferencia resultado único, lista de resultados, carregamento, ausência de resultados e erro. As respostas são verificadas antes da renderização; falhas de rede, JSON inválido e estruturas incompatíveis produzem mensagens controladas. O layout é responsivo, utiliza rótulos, regiões de status e alertas e segue como referência visual o Portal Brasileiro de Dados Abertos.

6 Estratégia e resultados dos testes

Tabela 2: Consolidação das verificações executadas.

Verificação	Resultado
Backend automatizado	12 de 12 testes aprovados.
API funcional e erros	20 de 20 cenários aprovados contra Flask, Gunicorn e PostgreSQL reais.
SQL injection	45 de 45 casos aprovados, sem alteração de tabelas, contagens ou fingerprints.
Carga de capacidade	20 requisições, concorrência 2, todas HTTP 200 e sem erro de transporte.
Sobrecarga	200 requisições, concorrência 32; 2 HTTP 200 e 198 HTTP 429, sem erro de transporte.
Disponibilidade posterior	`/health` respondeu HTTP 200 após a sobrecarga.
Timeout real	Consulta bloqueada retornou HTTP 504 em aproximadamente 2 segundos.
Clone limpo	Três imagens construídas; banco e backend saudáveis; frontend e `/health` com HTTP 200.
Integração do frontend	Uma busca percorreu frontend, proxy e API e apresentou o estado sem resultados.

A bateria funcional cobriu filtro vazio, espaços, tentativa representativa de injeção, nome inexistente, múltiplos resultados paginados, cargo inválido, UF inválida, órgão inválido, rota e página inexistentes, parâmetro desconhecido, limites inválidos, combinação inválida, timeout, concorrência e disponibilidade posterior. Na bateria de segurança, os payloads foram enviados aos cinco filtros. As quatro tabelas, suas contagens e seus fingerprints permaneceram iguais antes e depois dos 45 casos.

No cenário de capacidade, as 20 respostas HTTP 200 foram concluídas em 2,921 s, com 6,85 requisições por segundo, mediana de 276,08 ms e p95 de 367,76 ms. No cenário de sobrecarga, o mecanismo de concorrência recusou 198 requisições com HTTP 429 e permitiu duas respostas HTTP 200. Não houve erro de transporte e o endpoint de saúde continuou disponível.

Não houve cenário reprovado ou bloqueado nas evidências persistidas. Não foram executados teste prolongado de estabilidade, distribuição entre múltiplas instâncias, medição com o conjunto público completo nem o aplicativo desktop. Também não existe artefato

versionado de uma bateria manual completa do frontend. A evidência persistida comprova o build, o carregamento da interface e o caminho integrado do estado sem resultados.

7 Divisão diária das tasks

As Tabelas 3 e 4 usam a data em que a entrega entrou no histórico integrado ou foi registrada no acompanhamento. Dias sem entrega são mantidos de forma explícita.

Tabela 3: Registro diário da Sprint 1.

Data	Pedro	Marcos	Alex
12/09	INF-01/PROJ-3	-	-
13/09	INF-02/PROJ-4 (gestão, sem commit)	BD-01/PROJ-15; BD-02/PROJ-16	FRONT-01/PROJ-29; FRONT-02/PROJ-30
14/09	INF-03/PROJ-5	BD-03/PROJ-17	FRONT-03/PROJ-31
15/09	INF-04/PROJ-6	BD-04/PROJ-18	FRONT-04/PROJ-32
16/09	INF-05/PROJ-7	BD-05/PROJ-19	FRONT-05/PROJ-33
17/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
18/09	INF-06/PROJ-8; INF-07/PROJ-9	BD-06/PROJ-20	FRONT-06/PROJ-34; FRONT-07/PROJ-35
19/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
20/09	INF-08/PROJ-10; QA-01/PROJ-11	BD-07/PROJ-21; BD-08/PROJ-22; BACK-01/PROJ-23	FRONT-08/PROJ-36; FRONT-09/PROJ-37
21/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
22/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
23/09	QA-02/PROJ-12; LID-02/PROJ-14; atualização de LID-01/PROJ-13	BACK-02/PROJ-24; BACK-03/PROJ-25; BACK-04/PROJ-26	FRONT-10/PROJ-38
24/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
25/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
26/09	Conclusão de LID-01/PROJ-13	BACK-05/PROJ-27; BACK-06/PROJ-28	Sem entrega integrada

Tabela 4: Registro diário da Sprint 2.

Data	Pedro	Marcos	Alex
27/09	INF-09/PROJ-39	BACK-07/PROJ-51	FRONT-11/PROJ-61; FRONT-12/PROJ-62
28/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
29/09	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
30/09	INF-10/PROJ-40; INF-11/PROJ-41; INF-12/PROJ-42	BACK-08/PROJ-52; BACK-09/PROJ-53; BACK-10/PROJ-54	FRONT-13/PROJ-63; FRONT-14/PROJ-64; FRONT-15/PROJ-65
01/10	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada

Data	Pedro	Marcos	Alex
02/10	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
03/10	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
04/10	QA-03/PROJ-43 a QA-06/PROJ-46	BACK-11/PROJ-55; BACK-12/PROJ-56; BD-09/PROJ-57; BACK-13/PROJ-58	FRONT-16/PROJ-66 a FRONT-19/PROJ-69
05/10	Sem entrega integrada	Sem entrega integrada	Sem entrega integrada
06/10	LID-03/PROJ-47; LID-04/PROJ-48	BACK-14/PROJ-59; BACK-15/PROJ-60	FRONT-20/PROJ-70
07/10	Sem entrega integrada	Sem pendências obrigatórias	Sem pendências obrigatórias
08/10	Sem entrega integrada	Sem pendências obrigatórias	Sem pendências obrigatórias
09/10	LID-05/PROJ-49; LID-06/PROJ-50; LID-07/PROJ-72	Sem pendências obrigatórias	Sem pendências obrigatórias

8 Histórico de commits por integrante

O histórico foi conferido em `dev` até o merge `43667cc`, com os commits sem merge até `e5647e8`. A tarefa `INF-02/PROJ-4` consistiu na organização do backlog e do quadro no Jira, sem commit de código. A task `LID-07/PROJ-72` aparece como "este commit" porque o hash não pode ser incluído no próprio conteúdo sem alterar novamente esse hash.

8.1 Pedro

Data	Task/Jira	Commit	Resultado
12/09	INF-01/PROJ-3	82574ee; 0a50fb5	Estrutura inicial ajustada para a raiz.
14/09	INF-03/PROJ-5	c8395e6	Composição inicial criada.
15/09	INF-04/PROJ-6	9785ee7	Imagem do banco criada.
16/09	INF-05/PROJ-7	e4be7c3	Imagem do backend criada.
18/09	INF-06/PROJ-8	580e9ef	Imagem do frontend criada.
18/09	INF-07/PROJ-9	3baff68	Orquestração dos containers validada.
20/09	INF-08/PROJ-10	493c300	Variáveis de ambiente configuradas.
20/09	QA-01/PROJ-11	edf7e9d	Plano de testes da API definido.
23/09	QA-02/PROJ-12	b12a706	Casos de SQL injection definidos.
23 e 26/09	LID-01/PROJ-13	79ef587; 5236647	Acompanhamento da Sprint 1 registrado e encerrado.
23/09	LID-02/PROJ-14	26cd934	Estrutura do relatório final criada.
27/09	INF-09/PROJ-39	467b7a3	Aplicação integrada à composição.
30/09	INF-10/PROJ-40	fcc9fb3	Healthchecks e reinício configurados.
30/09	INF-11/PROJ-41	555a6a3	Buscas simultâneas limitadas.

Data	Task/Jira	Commit	Resultado
30/09	INF-12/PROJ-42	dd80b71	Timeout aplicado às consultas.
04/10	QA-03/PROJ-43	1274a0d	Teste de carga executado.
04/10	QA-04/PROJ-44	569da1f	Resultados e limitações da carga registrados.
04/10	QA-05/PROJ-45	98f891d	Cenários de erro executados.
04/10	QA-06/PROJ-46	49fe3e6	Proteção contra SQL injection validada.
06/10	LID-03/PROJ-47	3acd6e5	Divisão das tasks e histórico consolidados.
06/10	LID-04/PROJ-48	f2faee7	Burndown das duas sprints incluído.
09/10	LID-05/PROJ-49	cc1ebfa	Análise crítica da equipe registrada.
09/10	LID-06/PROJ-50	e5647e8	Inicialização por clone limpo validada.
09/10	LID-07/PROJ-72	este commit	Relatório revisado e consolidado em formato SBC.

8.2 Marcos

Data	Task/Jira	Commit	Resultado
13/09	BD-01/PROJ-15	1148065	Driver do PostgreSQL adicionado.
13/09	BD-02/PROJ-16	882f94d	Análise inicial dos datasets registrada.
14/09	BD-03/PROJ-17	d64f588	Campos necessários mapeados.
15/09	BD-04/PROJ-18	fe3b0d8	Schema do banco modelado.
16/09	BD-05/PROJ-19	286d627	Tabelas do banco criadas.
18/09	BD-06/PROJ-20	70c9196	Dados preparados para importação.
20/09	BD-07/PROJ-21	d27e820	Dados tratados importados.
20/09	BD-08/PROJ-22	66f5f0b	Índices de busca criados.
20/09	BACK-01/PROJ-23	e4513c3	Esqueleto Flask criado.
23/09	BACK-02/PROJ-24	ccdf5c4	Gunicorn multithread configurado.
23/09	BACK-03/PROJ-25	4cfa188	Conexão com PostgreSQL criada.
23/09	BACK-04/PROJ-26	70f63a5	Busca exata por nome implementada.
26/09	BACK-05/PROJ-27	8e440b9	Filtros por cargo e UF implementados.
26/09	BACK-06/PROJ-28	47e3872	Filtro por órgão implementado.
27/09	BACK-07/PROJ-51	f8a1064	Busca combinada implementada.
30/09	BACK-08/PROJ-52	ed9e38f	Busca por similaridade implementada.
30/09	BACK-09/PROJ-53	389c272	Paginação dos resultados implementada.
30/09	BACK-10/PROJ-54	728568f	Parâmetros de busca validados.
04/10	BACK-11/PROJ-55	545c692	Cenários de erro tratados.
04/10	BACK-12/PROJ-56	e1d822f	Respostas JSON padronizadas.
04/10	BD-09/PROJ-57	1404a59	Busca por similaridade otimizada.
04/10	BACK-13/PROJ-58	41eb8da	Testes automatizados dos endpoints adicionados.
06/10	BACK-14/PROJ-59	24a866c	Endpoints, parâmetros e exemplos documentados.
06/10	BACK-15/PROJ-60	0a28f98	Respostas de erro corrigidas e revalidadas.

8.3 Alex

Data	Task/Jira	Commit	Resultado
13/09	FRONT-01/PROJ-29	58ffbc9	Estrutura inicial configurada.
13/09	FRONT-02/PROJ-30	776d42f	Identidade visual definida.
14/09	FRONT-03/PROJ-31	9ab57fe	Layout base criado.
15/09	FRONT-04/PROJ-32	ea59923	Formulário de busca criado.
16/09	FRONT-05/PROJ-33	9febd91	Formulário de busca estilizado.
18/09	FRONT-06/PROJ-34	4123f17	Listagem com registros fictícios criada.
18/09	FRONT-07/PROJ-35	f6a31fd	Listagem de resultados estilizada.
20/09	FRONT-08/PROJ-36	8570123	Paginação com dados fictícios criada.
20/09	FRONT-09/PROJ-37	ad846df	Estados simulados de resultados criados.
23/09	FRONT-10/PROJ-38	44d97f8	Responsividade inicial ajustada.
27/09	FRONT-11/PROJ-61	cc9f180	Frontend integrado à API real.
27/09	FRONT-12/PROJ-62	3703715	Busca exata por nome implementada.
30/09	FRONT-13/PROJ-63	ef2df83	Busca por similaridade implementada.
30/09	FRONT-14/PROJ-64	4b13c89	Filtros individuais implementados.
30/09	FRONT-15/PROJ-65	fb65451	Busca combinada implementada.
04/10	FRONT-16/PROJ-66	a9ed8db	Paginação integrada aos resultados reais.
04/10	FRONT-17/PROJ-67	09b01a6	Resultado único diferenciado da listagem.
04/10	FRONT-18/PROJ-68	982d69a	Feedback de carregamento e erro adicionado.
04/10	FRONT-19/PROJ-69	70e40cd	Responsividade e apresentação final ajustadas.
06/10	FRONT-20/PROJ-70	6ab7768	Estados de busca e respostas inválidas corrigidos.

9 Burndown

O saldo real representa o fim de cada dia. A Sprint 2 mantém sua linha-base de 57 pontos e registra a mudança de escopo de 09/10 separadamente. Antes da conclusão havia 3 pontos restantes; a inclusão da PROJ-72 acrescentou 2 pontos; as três tasks do dia concluíram 5 pontos e levaram o saldo a zero.

Tabela 8: Fechamento das sprints.

Sprint	Base	Ajuste	Escopo final	Concluídos	Fechamento
Sprint 1	56	0	56	56	26/09
Sprint 2	57	+2	59	59	09/10

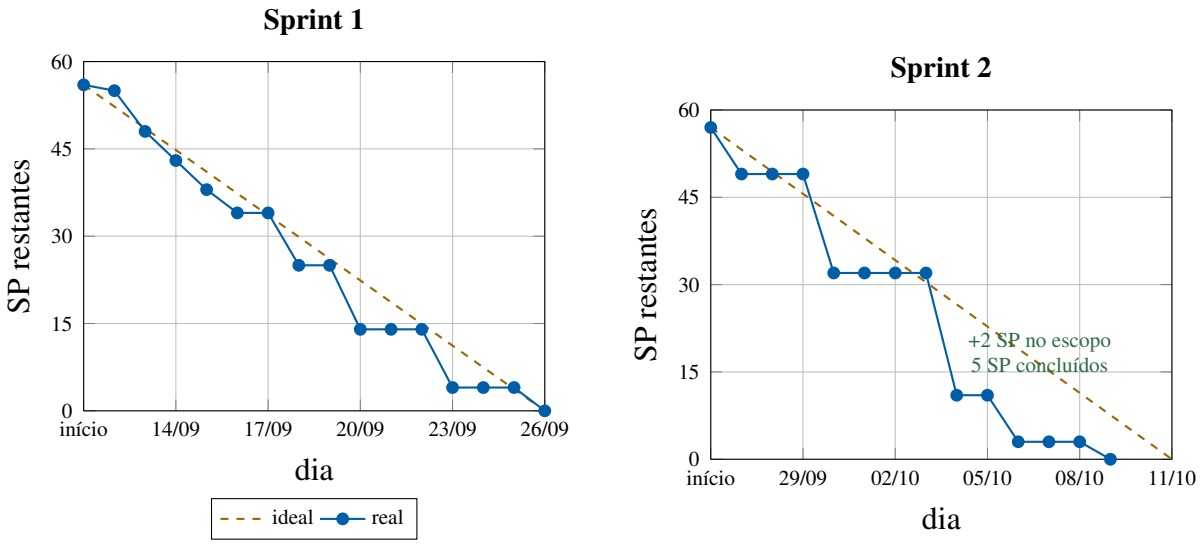


Figura 2: Story Points restantes. Sprint 2: linha-base de 57 SP e escopo final de 59 SP após o ajuste de 09/10.

10 Análise crítica da equipe

10.1 Pontos fortes

- A divisão por área permaneceu estável e o histórico preservou a autoria dos três integrantes.
- As tasks permaneceram identificadas por Jira mesmo quando atividades relacionadas compartilharam a mesma integração.
- As evidências reproduzíveis cobriram API, segurança, carga e clone limpo. Os problemas encontrados pelo QA geraram correções no backend e no frontend.

10.2 Pontos fracos e limitações

- A cadência não foi uniforme e houve quedas concentradas no burndown.
- A capacidade ficou limitada a duas buscas simultâneas. Na sobrecarga, 198 de 200 requisições foram rejeitadas com HTTP 429. O comportamento preservou a aplicação, mas evidencia baixa capacidade para picos.
- Os testes foram locais, com uma instância do backend e 100.000 registros sintéticos. Não houve estabilidade prolongada, distribuição entre instâncias ou medição com o conjunto público completo.
- A documentação final foi consolidada perto do encerramento, exigindo a conferência simultânea do board, acompanhamento e histórico Git.

10.3 Dificuldades e impedimentos

Período	Dificuldade ou impedimento	Ação e resultado
12/09	A estrutura inicial não correspondia à convenção da raiz.	Backend, frontend, banco e composição foram mantidos diretamente na raiz.
13/09	Três integrantes no mesmo computador precisavam manter identidades distintas.	Clones e identidades separados preservaram a autoria dos commits.
15/09	A validação das imagens dependia do Docker em execução.	O ambiente passou a ser inicializado antes das verificações.
17 a 25/09	Interrupções e dias sem integração acumularam entregas.	O trabalho foi retomado em lotes, com commits atômicos e identificação por task.
04/10	A carga confirmou baixa capacidade e custo da busca ampla por similaridade.	Limite de concorrência, timeout e índice de similaridade fizeram o excesso falhar de forma controlada.

11 Conclusão

O projeto entregou a aplicação web modularizada em três serviços, com PostgreSQL, API Flask executada pelo Gunicorn e frontend React integrado por HTTP. Foram implementadas buscas por nome exato, similaridade, cargo, UF, órgão e combinações, além de paginação e apresentação diferenciada para um ou vários resultados.

A API possui respostas JSON padronizadas, validação de entrada, consultas parametrizadas, limite de concorrência, timeout e tratamento dos erros previstos no enunciado. Os testes automatizados, funcionais, de segurança, carga e inicialização em clone limpo foram concluídos sem reprovações registradas.

A Sprint 1 encerrou com 56 de 56 pontos e a Sprint 2 com 59 de 59. Não restam atividades obrigatórias do backlog. O aplicativo desktop não foi desenvolvido por corresponder ao item adicional da avaliação.

Os arquivos públicos do banco não são versionados. Um clone limpo cria o schema e inicia os serviços, mas permanece sem registros até o tratamento e a importação. Permanecem como limitações os testes exclusivamente locais, a capacidade de duas buscas simultâneas e a ausência de testes prolongados ou distribuídos.

12 Evidências e referências

1. Portal Brasileiro de Dados Abertos. Gestão de Pessoas do Executivo Federal - Aposentados.
2. Portal Brasileiro de Dados Abertos. Gestão de Pessoas do Executivo Federal - Carreiras/Cargos.
3. Dicionário de dados de Aposentados.
4. Dicionário de dados de Carreiras/Cargos.
5. Documentação versionada: análise dos datasets, mapeamento de campos, modelo do banco, referência da API, plano de testes, casos de SQL injection, validação do Compose, acompanhamento e burndown.
6. Evidências versionadas em `docs/evidencias/`: validação funcional da API, SQL injection, carga de capacidade e sobrecarga, executadas em 04/10/2026.
7. Histórico Git integrado em `dev`, usado para conferir autoria, datas e mensagens dos commits.